

CONTROLLED EXECUTION SANDBOX

Submitted by:
Jhanvi Patel

TABLE OF CONTENTS

1. Introduction
2. Task: Controlled Execution Sandbox
3. Implementation Details
4. Why Python Was Chosen
5. Results and Observations
6. Conclusion
7. References

1. Introduction

Cybersecurity is an important part of protecting systems, data, and applications from unauthorized access and harmful activities. As we rely more on digital platforms in everyday life, it becomes essential to ensure that data remains secure and that systems behave as expected.

In this assignment, a key cybersecurity concepts are implemented: a controlled execution sandbox. The controlled execution sandbox focuses on restricting user inputs and preventing unsafe or potentially harmful operations from being executed.

This task was developed using Python because of its simplicity and the availability of useful built-in libraries. This made it easier to demonstrate important security concepts such as hashing, input validation, and controlled execution in a clear and practical way.

Overall, this assignment helped in understanding how basic security mechanisms can be applied in real-world scenarios. It also highlights the importance of building systems that are not only functional but also secure and reliable.

2. Task: Controlled Execution Sandbox

2.1 Objective

The objective of this task is to design a system that can safely execute user-provided input while preventing any unsafe or malicious operations. The goal is to create a controlled environment where only permitted actions are allowed, and any suspicious or restricted operations are detected and blocked.

2.2 System Design

This system is based on the concept of sandboxing, where user input is executed in a restricted environment to prevent unauthorized access to system resources. By limiting available operations and blocking unsafe commands, the system ensures that only safe and controlled execution is allowed. This helps in reducing security risks and maintaining system stability.

The system works by analysing user input before execution. Instead of directly running all inputs, it first checks for potentially dangerous keywords such as system access commands, file operations, or code execution functions.

A filtering mechanism is used to identify different types of threats, including system access, file access, code execution, and network-related operations. If any such activity is detected, the system blocks the command and provides a clear warning message.

Additionally, the execution environment is restricted by disabling built-in functions, ensuring that the user cannot access sensitive system-level features. Input size is also limited to prevent abnormal or excessive operations.

The system also maintains a simple log of user inputs and their outcomes, allowing monitoring of all actions performed during a session.

2.3 Implementation

The sandbox was implemented using Python. The program continuously accepts user input and processes it through a validation function. This function checks for restricted keywords and classifies the type of potential threat.

If the input is considered safe, it is executed in a restricted environment using a controlled evaluation method. If the input is unsafe, it is blocked, and an appropriate message is displayed indicating the type of violation.

A session history feature was also implemented, which records all user inputs along with their results. This helps in understanding how the system handled different types of inputs.

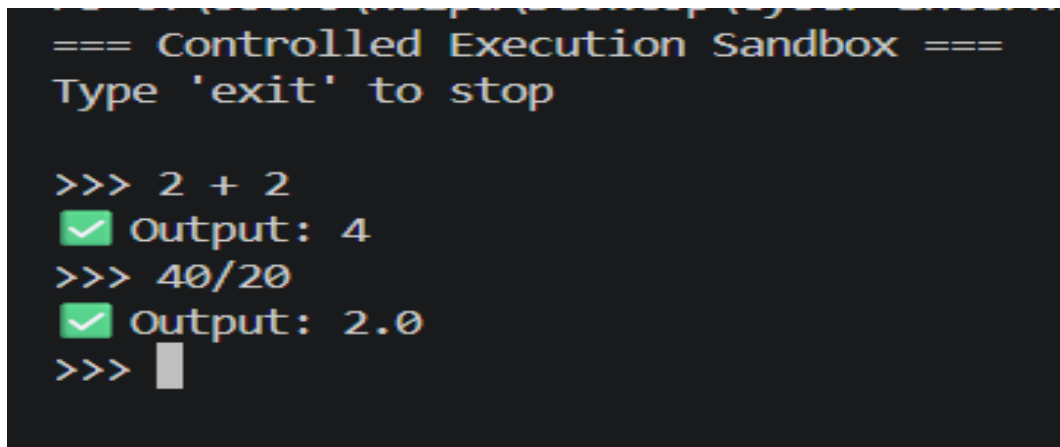
2.4 Results and Output

The system was tested with various inputs to evaluate its behavior. Simple arithmetic expressions were executed successfully, demonstrating normal functionality. Attempts to perform restricted actions such as accessing system modules or files were blocked with clear messages.

Invalid inputs were also handled gracefully, ensuring that the system does not crash or behave unexpectedly. At the end of the session, a summary of all executed and blocked commands was displayed.

The following screenshots demonstrate the working of the sandbox:

(Figure 1: Safe Execution of Input)



```
=== Controlled Execution Sandbox ===  
Type 'exit' to stop  
  
>>> 2 + 2  
✓ Output: 4  
>>> 40/20  
✓ Output: 2.0  
>>> 
```

(Figure 2: Blocking System Access Attempt)

```
>>> import os  
✖ Blocked: system access attempt detected  
>>> █
```

(Figure 3: Blocking File Access Attempt)

```
>>> open("file.txt")  
✖ Blocked: file access attempt detected  
>>> █
```

(Figure 4: Handling Invalid Input)

```
>>> abc  
⚠ Invalid or not allowed  
>>> █
```

(Figure 5: Session Summary)

```
PS C:\Users\helpa\Desktop\Cyber Internship Assignment> python -u "c:\Users\helpa\Desktop\Cyber Internship Assignment\task2.py"
=== Controlled Execution Sandbox ===
Type 'exit' to stop

>>> 2 + 2
✔ Output: 4
>>> 40/20
✔ Output: 2.0
>>> import os
✖ Blocked: system access attempt detected
>>> open("file.txt")
✖ Blocked: file access attempt detected
>>> abc
⚠ Invalid or not allowed
>>> exit

Session Summary:
- 2 + 2 → ✔ Output: 4
- 40/20 → ✔ Output: 2.0
- import os → ✖ Blocked: system access attempt detected
- open("file.txt") → ✖ Blocked: file access attempt detected
- abc → ⚠ Invalid or not allowed
PS C:\Users\helpa\Desktop\Cyber Internship Assignment> |
```

3. Implementation Details

This Task was implemented using Python due to its simplicity and flexibility. The controlled execution sandbox was implemented by restricting user input and executing it in a limited environment. Unsafe keywords were filtered, and execution was performed using a restricted evaluation method that disables built-in functions. Additional checks such as input length validation and session logging were also included to improve control and monitoring.

4. Why Python Was Chosen?

The implementation focuses on clarity and correctness, making it easy to understand how core security concepts are applied in practice.

Python was selected for this assignment because of its readability and ease of use, which makes it suitable for implementing security concepts in a clear and understandable way.

Additionally, Python allows quick development and testing of ideas, which is helpful when building systems like logging mechanisms and controlled execution environments. Its simple syntax makes it easier to focus on logic rather than complex language structures.

Overall, Python was an appropriate choice for demonstrating the required concepts effectively.

5. Results & Observations

The implemented system was tested with multiple scenarios to evaluate their effectiveness. In In Task, the sandbox environment allowed safe execution of valid inputs while blocking unsafe operations such as system access and file handling. The system also classified different types of threats and provided clear feedback to the user.

6. Conclusion

Overall, the system performed as expected and demonstrated the importance of implementing security mechanisms in software systems.

This assignment provided practical experience in implementing core cybersecurity concepts. The tamper-evident logging system demonstrated how data integrity can be maintained using cryptographic hashing, while the controlled execution sandbox highlighted the importance of restricting unsafe operations.

Through this implementation, it became clear that even simple systems can be designed to detect and prevent potential security risks. The assignment also emphasized the importance of writing secure and reliable code.

This assignment was completed through self-learning using various online resources such as documentation, tutorials, and cybersecurity articles. It gave me the opportunity to explore new concepts and apply them in a practical way. While working on this, I was able to develop a clearer understanding of important topics like data integrity, hashing, and controlled execution. This assignment also reflects my ability to learn independently and implement concepts effectively. I am motivated to keep improving my skills and continue learning more in the field of cybersecurity and software development.

Overall, this work helped in understanding how theoretical security concepts can be applied in real-world scenarios.

7. References

1. Python hashlib documentation

<https://docs.python.org/3/library/hashlib.html>

2. Python official documentation

<https://docs.python.org/3/>

3. OWASP – Secure Coding Practices

<https://owasp.org/www-project-secure-coding-practices/>

4. Python eval() function explanation

<https://www.programiz.com/python-programming/methods/built-in/eval>

5. Basic Cybersecurity Concepts (IBM)

<https://www.ibm.com/topics/cybersecurity>

